SafeRoute Guardian — Product Requirements Document

Team Zephyr · Hackering 2.0 (Open Innovation) · Kavya Rajesh Iyer (Captain), Pranjal Singh Thakur, Varsha G, Vikas K P

Status: Draft v1 (hackathon MVP) · Items marked *(proposed)* are assumptions added to fill gaps in the pitch deck and should be confirmed by the team.

1. Overview

SafeRoute Guardian is a context-aware safety layer for navigation. Existing navigation apps optimise for time and distance. SafeRoute Guardian treats safety as a first-class routing objective: it evaluates origin, destination, route geometry and time of travel together, flags risky segments on the fastest route, and instantly offers a safer corridor with a clear time trade-off.

Tagline: Navigation that asks "Is this route safe?" — not only "How fast can I get there?"

Core routing idea: Route Cost = Travel Time + Risk Penalty.

Headline demo scenario: Fastest route is 18 min with a Safety Score of 58/100 and crosses a high-risk zone. The safe corridor is 21 min with a Safety Score of 94/100 (+3 min).

2. Problem Statement

- Navigation optimises for time and distance, not safety; safety context sits outside the routing decision.
- The fastest route may pass poorly lit or isolated segments.
- Risk is discovered only after the journey begins.
- Users get warnings, not actionable alternatives.
- Static safety data ignores time of travel.
- No safer alternative is offered at the moment of route selection.
- Late-night travel amplifies environmental risk, yet a small time trade-off can mean a significantly safer path.

3. Target Users

Persona	Context	Primary need
Women travelling alone at night	Late-evening commutes, returning home	A safer route before they start moving
Pedestrians in unfamiliar areas	Visitors, new residents	Awareness of risky segments they cannot judge themselves
Students	Campus, library, PG and hostel commutes	Quick, low-effort safer alternative
Night-shift workers and solo commuters	Recurring late-hour travel	Repeatable, trustworthy safe routing

4. Goals and Non-Goals

Goals (MVP)

1. Let a user enter origin, destination and time of travel and see the fastest route plus a safer alternative.
2. Detect where each candidate route intersects risk zones and quantify it as a Safety Score (0–100).
3. Present the time versus safety trade-off clearly and allow a one-tap switch.
4. Provide an AI safety copilot that explains the risk in plain language.
5. Demonstrate an SOS workflow end to end as a simulation.

Non-Goals (MVP)

- Real emergency services integration. SOS is a simulated workflow only.
- Real crime or incident data. The demo uses synthetic Bengaluru GeoJSON risk zones.
- Turn-by-turn live navigation, offline maps or native mobile apps.
- User accounts, community reporting, or crowd-sourced data ingestion *(future)*.
- Coverage outside the Bengaluru demo area.

5. User Stories

ID	As a…	I want to…	So that…
US-1	Solo traveller	enter where I am going and when	the route reflects the time of day
US-2	Solo traveller	see which parts of the fastest route are risky	I understand why it is flagged
US-3	Solo traveller	see a safer option with its extra minutes	I can decide knowingly
US-4	Solo traveller	switch to the safer route in one tap	I do not need to re-plan manually
US-5	Solo traveller	ask an assistant why a route is risky or what to do	I get plain-language guidance
US-6	Solo traveller	trigger an SOS flow	I see how help would be summoned

6. Functional Requirements

6.1 User Input

ID	Requirement	Priority
FR-1	User can set origin and destination via address search and map click.	P0
FR-2	User can specify time of travel (default: now).	P0
FR-3	Inputs are validated; an inline error is shown for unroutable or out-of-area points.	P1

6.2 Map and Routing

ID	Requirement	Priority
FR-4	Render an interactive map canvas (Mapbox GL JS).	P0
FR-5	Fetch the fastest route and candidate alternatives from the Mapbox Directions API.	P0

ID	Requirement	Priority
FR-6	Generate at least one candidate safer route, for example via alternatives or waypoints that steer around high-risk zones *(proposed)*.	P0
FR-7	Draw routes distinctly, with risky segments highlighted in the route colour scheme.	P0

6.3 Safety Zones and Spatial Analysis

ID	Requirement	Priority
FR-8	Load synthetic GeoJSON risk zones for Bengaluru, classified Red (High), Yellow (Moderate) and Green (Lower).	P0
FR-9	Use Turf.js to detect intersections between each route and each zone, and compute length of exposure per risk class.	P0
FR-10	Show the risk zones as a toggleable map overlay.	P1
FR-11	Zone risk can vary by time of day (for example, night multiplier) so the same segment scores differently at different hours.	P0

6.4 Risk Engine

ID	Requirement	Priority
FR-12	Compute a Safety Score (0–100) per route from risk exposure.	P0
FR-13	Compute Route Cost = Travel Time + Risk Penalty for every candidate and rank by cost.	P0
FR-14	Select the lowest-cost route as the safe corridor and compare it against the fastest route.	P0
FR-15	Expose the scoring weights as configuration so they can be tuned during the demo *(proposed)*.	P2

6.5 Output and Interaction

ID	Requirement	Priority
FR-16	Comparison card shows duration, Safety Score and risk-zone crossings for fastest versus safe corridor, including the time delta (for example, +3 min).	P0
FR-17	One-tap Safe Switch changes the active route and updates the map and card instantly.	P0
FR-18	Gemini-powered safety copilot answers questions about the selected routes, grounded in the computed scores and zone data.	P1
FR-19	SOS workflow simulator: a clearly labelled demo that walks through alert, location share and contact notification steps without contacting anyone.	P1

7. Risk Scoring Model

Principle: Route Cost = Travel Time + Risk Penalty.

(proposed parameters, to be tuned)

- **Zone weights:** Red = 3, Yellow = 1.5, Green = 0.5.
- **Time-of-day multiplier:** daytime 1.0, evening 1.3, late night 1.6.
- **Risk Penalty (minutes-equivalent):** sum over zone classes of (exposed length in km × zone weight × time multiplier × penalty factor).
- **Safety Score:** 100 minus normalised exposure, clamped to 0–100, so a route that avoids Red zones scores in the 90s and one crossing a Red zone scores in the 50s.
- **Selection rule:** choose the candidate with the lowest Route Cost. Show the safer route only if its score is meaningfully higher than the fastest route's *(proposed threshold: 10 points)*, otherwise confirm the fastest route is already safe.

The demo must be reproducible: with the seeded Bengaluru data and the chosen demo pair, the engine yields 18 min / 58 for the fastest and 21 min / 94 for the safe corridor.

8. User Flow

1. User opens the app and enters origin, destination and time of travel.

2. System requests routes from Mapbox Directions and generates candidate safer routes.
3. Turf.js checks every route against the GeoJSON risk zones.
4. Risk engine computes Safety Scores and Route Cost.
5. UI shows the fastest route flagged with its risky segment, and the safe corridor beside it.
6. User taps Safe Switch to adopt the safer route.
7. User optionally asks the Gemini copilot for explanation or advice, or runs the SOS simulation.

9. Architecture and Tech Stack

Layer	Technology	Responsibility
Frontend	React, Tailwind CSS	UI, state, comparison card, copilot panel
Mapping	Mapbox GL JS, Directions API	Map canvas, route fetching
Geospatial	Turf.js, GeoJSON	Intersection detection, exposure length
AI	Gemini API	Safety copilot responses
Backend	Node.js, Express	API proxy for Gemini and Mapbox keys, scoring endpoint, config

Design notes:

- Keep API keys server-side; the frontend calls the Express backend for Gemini requests.
- Risk computation may run client-side for speed in the demo, with the backend as the source of truth for configuration *(proposed)*.
- The copilot prompt receives structured route data (durations, scores, zones crossed, time) so answers stay grounded and do not invent safety claims.

10. Data Requirements

- Synthetic GeoJSON polygons for Bengaluru, each with properties: id, name, risk class (red/yellow/green), base weight, and optional per-time-band modifiers *(proposed)*.
- At least one demo origin–destination pair that crosses a Red zone on the fastest route and has a viable detour.

- All demo data must be labelled as synthetic in the UI and in the pitch to avoid implying real crime statistics.

11. Non-Functional Requirements

Area	Requirement
Performance	Routes and scores appear within about 3 seconds of submitting a query; Safe Switch updates in under 300 ms.
Usability	Primary flow completable in under three taps/clicks after input; responsive layout for mobile browsers.
Reliability	Graceful fallback if Directions API or Gemini fails (show routes without copilot, clear error message).
Security	No API keys in the client bundle; rate-limit the backend; no storage of personal location data in the MVP.
Privacy	Location is used only for the active session; SOS simulation never transmits real contacts or messages.
Transparency	UI states that scores are based on synthetic demo data and are not a guarantee of safety.

12. Success Metrics

Hackathon demo:

- Live demo completes without failure, showing the 18 min / 58 versus 21 min / 94 comparison.
- Judges can see the time-of-day effect by changing the time input and observing a different score.
- Copilot gives a correct, grounded explanation on at least three sample questions.

Post-MVP product *(proposed)*:

- Percentage of risky-route queries where the user adopts the safe corridor.
- Average added minutes accepted per switch.
- Reduction in high-risk segment exposure per journey.

13. Risks and Mitigations

Risk	Impact	Mitigation
Synthetic data mistaken for real safety data	Misleading users and judges	Label clearly; position as a proof of concept
Safer route generation finds no meaningful alternative	Weak demo	Pre-select demo pairs; use waypoint-based detours
Mapbox or Gemini outage or quota limits	Demo failure	Cache a demo route set; fallback messaging
Over-reliance on a score creating false confidence	User harm in real use	Disclaimers; avoid absolute language ("safer", not "safe")
Scoring tuned to force the headline numbers	Credibility	Document weights; keep config transparent

14. Milestones *(proposed)*

Phase	Deliverable
1	Map canvas, input form, Directions API integration
2	GeoJSON zones, Turf.js intersection, Safety Score
3	Route Cost engine, safe corridor generation, comparison card, Safe Switch
4	Gemini copilot via Express backend, SOS simulator
5	Time-of-day modifiers, polish, demo rehearsal, fallback data

15. Future Scope

- Real data sources: government and public datasets, street-light coverage, footfall, open-business density.
- Community incident reporting and verification.
- Live navigation with in-journey re-routing when conditions change.
- Real SOS with trusted contacts and emergency services.
- Expansion to additional cities and a public safety-aware routing API.

16. Open Questions

1. Exact scoring weights and the formula for normalising exposure into the 0–100 Safety Score.
2. Method for producing candidate safer routes: Directions alternatives, avoid-polygon waypoints, or a custom graph search.
3. Whether risk evaluation runs client-side, server-side, or both.
4. Which Bengaluru neighbourhoods and demo origin–destination pairs to use.
5. How to present the copilot's guidance responsibly, given its advice depends on synthetic data.